

KLEPSYDRA AI ONNX IMPORTER GUIDE V25.5.5

08.02.2026

Overview	3
Technical requirements	3
Introduction	4
Klepsydra AI ONNX API	4
Klepsydra AI ONNX Support	4
Supported Layers	4
Background - Best Practices	6
Static vs dynamic input size	6
Batch Size	6
Data Format : Channel first (NCHW) vs Channel last (NHWC)	6
Post-Processing layers	6
Quantized layers	6
Exporting trained models	7
TensorFlow models	7
Quantized networks with TensorFlow	8
Pytorch models	8
Post-Export Steps	9
Quantized Layer output shapes	9
Simplification	9
Pruning unsupported layers	10
Testing	11

Overview

As the demand for onboard AI processing in space continues to grow—driven by applications such as autonomous navigation, payload data analysis, and anomaly detection—organizations face increasing pressure to deploy high-performance, mission-adaptable solutions that meet strict reliability and resource constraints.

Klepsydra AI is a next-generation deep neural network engine specifically designed for edge computing in space environments. Unlike traditional AI frameworks, Klepsydra AI delivers up to 4× higher data throughput through a unique two-dimensional parallelization strategy, optimized for low-latency and deterministic performance across a wide range of space-qualified processors and hardware accelerators.

The ONNX (Open Neural Network Exchange) file format is an open standard for representing machine learning and deep learning models, enabling interoperability between frameworks like PyTorch and TensorFlow. It provides a consistent way to describe model architecture, weights, and operations, allowing models to be trained in one environment and deployed in another. ONNX files also include metadata such as the OPSET version, which defines the set of supported operations, and can differ in data layout formats like channel-first (NCHW) or channel-last (NHWC), which may affect compatibility with different platforms.

For Klepsydra AI inference engine, supporting ONNX is essential to ensure broad compatibility with a wide range of pre-trained models and development workflows. This simplifies integration with existing AI pipelines and allows developers to deploy models across various devices without needing custom conversions. Support for ONNX, ensures the engine can handle diverse models reliably and consistently.

This guide explains how to export a deep neural network (DNN) from frameworks such as PyTorch or TensorFlow into the ONNX format, ensuring compatibility with Klepsydra's inference engine.

Technical requirements

As explained in AppendixB of the Getting Started with Klepsydra AI guide, the Protobufs library version 3.19.0 must be installed on the target computer¹. Follow the steps below to complete the installation:

- Go to some local folder in the target processor (e.g., \$HOME/thirdparties):

```
sudo apt install autoconf libtool
git clone https://github.com/protocolbuffers/protobuf.git
cd protobuf
git checkout v3.19.0
git submodule update --init --recursive
./autogen.sh && ./configure
make
sudo make install
```

- Optional:

```
cd ../
rm -rf protobuf
```

Users wishing to use OpenCV or Python bindings must install these dependencies manually (e.g., `apt install python3`). As with the previous section, this is only required for binary builds; Docker-based setups include these by default.

¹ Only necessary for binary builds. Not required for Docker-based setups.

Introduction

Klepsydra AI ONNX API

The Klepsydra AI API for ONNX files is straightforward: the ONNX model is passed as input to the `OnnxDNNImporter` API, as demonstrated in the example below.

```
std::vector<std::shared_ptr<kpsr::ai::Backend>> extensionBackends(0);
std::unique_ptr<kpsr::ai::DeepNeuralNetworkFactory> factory =
    kpsr::ai::OnnxDNNImporter::createDNNFactory(extensionBackends,
                                                onnxFile, configurationFile);
std::shared_ptr<kpsr::ai::DeepNeuralNetwork> kpsrNetwork = factory->getDNN();

auto saveToFileCallback = [&](const unsigned long &id, const std::vector<float>
&result) {
    std::cout << "Got result for input with id: " << id << std::endl;
};
kpsrNetwork->setCallback(saveToFileCallback);

std::vector<float> inputImage(inputShapeSize.size(), 0.0f);
auto id = kpsrNetwork->predict(inputImage);
```

Klepsydra AI also supports its own proprietary model format, called KPSR, which is slightly smaller than ONNX and does not rely on third-party tools like Protobufs for parsing. This format is designed to run efficiently on a wide range of processors and operating systems where ONNX parsing may not be supported. As part of the Klepsydra software package, two tools are included:

- A ONNX-to-KPSR conversion tool, which supports the same ONNX format as the main API.
- A KPSR-to-KPSR conversion tool, designed for environments where number types (e.g., `uint8`) are represented differently than on typical Linux-based ARM or x86 platforms.

Klepsydra AI ONNX Support

Klepsydra AI can load models in the ONNX format, with support for opset up to version 12.

Supported Layers

Models containing the following layers are supported:

- Abs
- Add
- AveragePool
- BatchNormalization
- Cast
- Clip
- Concat
- Constant
- Conv

- ConvTranspose
- DequantizeLinear
- Div
- Dropout
- Elu
- Flatten
- Gemm
- GlobalAveragePool
- InstanceNormalization
- LSTM
- LeakyRelu
- MatMul
- MaxPool
- Mul
- PRelu
- Pad
- QGemm
- QLinearAdd
- QLinearAveragePool
- QLinearConcat
- QLinearConv
- QLinearGlobalAveragePool
- QLinearLeakyRelu
- QLinearMatMul
- QLinearMul
- QLinearSigmoid
- QuantizeLinear
- Relu
- Reshape
- Resize
- Shape
- Sigmoid
- Slice
- Softmax
- Squeeze
- Sub
- Sum
- Tanh
- Transpose

Models can be trained using any framework, provided they can be exported to a valid ONNX format. However, since ONNX may represent the same operations using different layer types, it is recommended to follow the best practices outlined in this document to ensure compatibility and consistency.

Background - Best Practices

Static vs dynamic input size

ONNX format supports networks with dynamic input size. However, as Klepsydra AI pre-allocates memory while loading the network and before any predictions are performed, it is essential that the expected input size of data is known. Setting the input size while creating the ONNX representation additionally allows redundant layers to be removed.

Batch Size

Klepsydra AI does not do batched inference. So the batch size is always set to 1. In the ONNX representation this can simplify the network graph.

Data Format : Channel first (NCHW) vs Channel last (NHWC)

ONNX models represent data in the Channel first model. However, Klepsydra AI requires that data is in the Channel Last format (NHWC). As inputs and outputs are expected to be in channel last format, conversion of the DNN to ONNX may introduce Transpose layers in some cases. Klepsydra AI intelligently ignores these layers to avoid unnecessary data manipulation.

The data format is essential to be kept in mind when comparing the prediction obtained from Klepsydra AI to that obtained from other ONNX based inference engines.

Post-Processing layers

Although the post-processing steps of object detection networks like YOLO (and its variants) can be represented in ONNX, these layers involve simple operations which end up being represented as complicated graphs in the ONNX format. As a result, it is best to split the post-processing steps out of the DNN.

Quantized layers

The Opset 12 of the ONNX format has limited support for the Quantized layers. However, the ONNX Runtime accelerator provides specifications for quantized layers, compatible with the ONNX format, which are supported by Klepsydra AI. The following quantized layers have their definitions taken from ONNX Runtime's Operator set:

- QGemm
- QLinearAdd
- QLinearAveragePool
- QLinearConcat
- QLinearGlobalAveragePool
- QLinearLeakyRelu
- QLinearMul
- QLinearSigmoid

As these layers are not present in the ONNX spec for Opset 12, standard ONNX parsing tools may not be able to provide shape inference even if the input size of the network is set to a static value.

Exporting trained models

TensorFlow models

TensorFlow models (Keras and TFLite) can be converted to ONNX using the [tf2onnx](#) tool should ideally be saved in the Keras format

This Python tool requires that TensorFlow is also installed in your environment. TensorFlow models generally expect data in the Channel Last format, similar to Klepsydra AI. As ONNX format has data in the Channel First format, the TF to ONNX conversion may introduce Transpose layers as the first and last layers. The tf2onnx tool provides a way to avoid adding these layers.

The tf2onnx tool can be used with SaveModel, Keras models and TFLite format. For the example code presented below, we take the MobilenetV2 model from the [Model Garden for TensorFlow](#). The code below was tested with Python 3.10:

```
import TensorFlow as tf
import TensorFlow_models as tfm

bn_model = tfm.vision.backbones.MobileNet()
model = tfm.vision.models.ClassificationModel(bn_model, num_classes=1000)
model.save('mobilenetv2')
```

- Dummy export : If the input and output layer names are already known, this step can be skipped.

From the shell, call the converter on the model to obtain the input and output layer names.

```
python3 -m tf2onnx.convert --saved-model mobilenetv2 --output
mobilenetv2_tf.onnx --opset 12
```

You will get an output similar to this:

```
INFO - Using opset <onnx, 12>
INFO - Computed 0 values for constant folding
INFO - Optimizing ONNX model
INFO - After optimization: BatchNormalization -52 (52->0), Const -281 (349->68),
GlobalAveragePool +1 (0->1), Identity -2 (2->0), ReduceMean -1 (1->0), Reshape
-17 (17->0), Squeeze +1 (0->1), Transpose -224 (225->1)
INFO -
INFO - Successfully converted TensorFlow model mobilenet2 to ONNX
INFO - Model inputs: ['input_2']
INFO - Model outputs: ['dense']
INFO - ONNX model is saved at mobilenetv2_tf.onnx
```

Note the final three lines which provide the Model inputs and Model outputs.

- Re-export with correct options

Call the converter with the correct options to prevent NCHW -> NHWC conversion layers:

```
python3 -m tf2onnx.convert --saved-model mobilenetv2 --output  
mobilenetv2_tf.onnx --opset 12 --inputs-as-nchw 'input_2' --outputs-as-nchw  
'dense'
```

Note that if the model has multiple outputs, the output names should be comma separated and without any spaces between them.

Quantized networks with TensorFlow

Note that the tf2onnx [does not support the quantized layers](#) mentioned in [Section 1](#). As a result, on exporting a quantized TensorFlow model to ONNX, you will get a network which has a DequantizeLinear and QuantizeLinear around each floating point layer in the network. This format, known as the QDQ format, is not supported by Klepsydra AI. The network must be converted to QOperator format. Conversion to QOperator from QDQ format is out of the scope of this document. [ONNX Runtime's graph optimization](#) may help in this case.

Pytorch models

Pytorch provides [an API to export models](#) directly to ONNX. However, it is important to note that unlike TensorFlow, the trained model cannot be converted to ONNX without the original python code that describes the model.

The export function needs dummy input whose dimensions match the dimensions expected by the ONNX network (in NCHW, i.e. Channel first format).

Since pytorch and ONNX both use data in the Channel first format, no extraneous Transpose layers are added during the export to ONNX.

We provide an example for exporting the Mobilenetv2 available in the [TorchVision](#) package. If the procedure is performed in a docker container, you need to add the credentials and environment variable for the torch hub. The following commands are required inside your docker container:

```
sudo apt install ca-certificates  
sudo update-ca-certificates --fresh  
export SSL_CERT_DIR=/etc/ssl/certs
```

In Python you can run the following commands:

```
import torch  
  
mobilenet_model = torch.hub.load("pytorch/vision:release/0.14",  
model="mobilenet_v2")
```

We define the dummy input here to have the size 224x224 pixels with 3 channels. (The default model downloaded from torchvision has this as the input size).

```
x = torch.randn(1, 3, 224, 224)  
  
torch.onnx.export(mobilenet_model, x, "mobilenet_pytorch_tf.onnx",  
opset_version=12)
```

Post-Export Steps

Quantized Layer output shapes

In case the network has quantized layers from the ONNX Runtime, the ONNX python API cannot infer output shapes of the layers, even if using the [onnx.shape_inference](#). One potential solution is to manually add this information to the ONNX file. The Python API allows you to set the attributes of layer outputs manually. See <https://onnx.ai/onnx/repo-docs/IR.html#graphs> and https://onnx.ai/onnx/api/helper.html#onnx.helper.make_tensor_type_proto to set the `value_info` field of the ONNX model's GraphProto message.

Simplification

After the network has been exported to ONNX format, it is recommended to use the [ONNX Simplifier](#) to simplify the ONNX graph and potentially reduce the size of the network.

In case the network has quantized layers without shape information for the output layers, then the network will not be simplified efficiently.

For the two example mobilenet models presented about we show how simplification affects the respective models:

Simplifying model obtained from TensorFlow

The model exported from TensorFlow accepts 3 channel inputs of any dimensions. We define the input to have the same size as the Pytorch model in the above examples. The command to run and the output obtained is presented below:

```
onnxsim --overwrite-input-shape="1,3,224,224" mobilenetv2_tf.onnx
mobilenetv2_tf_sim.onnx
```

Simplifying...

Finish! Here is the difference:

	Original Model	Simplified Model
Add	10	10
Clip	35	35
Constant	68	68
Conv	52	52
GlobalAveragePool	1	1
MatMul	1	1
Squeeze	1	1
Model Size	13.3MiB	13.3MiB

Simplifying model obtained from pytorch

```
onnxsim --overwrite-input-shape="1,3,224,224" mobilenet_pytorch.onnx
mobilenetv2_pytorch_sim.onnx
```

Simplifying...

Finish! Here is the difference:

	Original Model	Simplified Model
Add	10	10
Clip	35	35
Constant	137	69
Conv	52	52

Flatten	1	1
Gemm	1	1
GlobalAveragePool	1	1
Identity	39	0
Model Size	13.3MiB	13.3MiB

Pruning unsupported layers

After simplification of the network, it may stay be the case that the network contains unsupported layers, or as in the case of the Yolo models, contains post-processing layers which should be removed. This can be done by using the sub-model [extract API](#).

Testing

The final exported network can then be tested for compatibility with Klepsydra AI by running the `kpsr_ai_onnx_compatible_checker_app`.

The model obtained from TensorFlow:

```
kpsr_ai_onnx_compatible_checker_app -m mobilenetv2_tf_sim.onnx
```

```
Input file provided:    mobilenetv2_tf_sim.onnx
```

```
---- Analysis started. ----z
```

```
Analysis ended.
```

```
---- Layer summary ----
```

```
LayerFactory: KlepsydraBackend
```

```
LayerFactory: QuantizedKpsrBackendFactory
```

```
LayerFactory: QuantizedInt8KpsrBackendFactory
```

```
LayerFactory: FullBackendFactory
```

```
- Add
```

```
- Clip
```

```
- Conv
```

```
- MatMul
```

```
- GlobalAveragePool
```

```
LayerFactory: QuantizedXBackendFactory
```

```
These layers are performed regardless of backends:
```

```
- Squeeze
```

And the model obtained from Pytorch:

```
kpsr_ai_onnx_compatible_checker_app -m mobilenetv2_pytorch_sim.onnx
```

```
Input file provided:    mobilenetv2_pytorch_sim.onnx
```

```
---- Analysis started. ----
```

```
Analysis ended.
```

```
---- Layer summary ----
```

```
LayerFactory: KlepsydraBackend
```

```
LayerFactory: QuantizedKpsrBackendFactory
```

```
LayerFactory: QuantizedInt8KpsrBackendFactory
```

```
LayerFactory: FullBackendFactory
```

```
- Add
```

```
- Clip
```

```
- Conv
```

```
- Gemm
```

```
- GlobalAveragePool
```

```
LayerFactory: QuantizedXBackendFactory
```

```
These layers are performed regardless of backends:
```

```
- Flatten
```